

Assignment 1

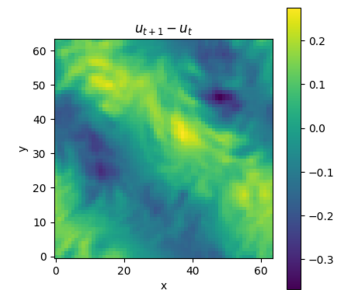
Autoregressive Simulation

Deadline: Friday September 25th, 23:59

Design and implement two types of neural surrogates, one for an Eulerian system and one for a Lagrangian system. The practicals (1 and 2) covering this material and the required dataset (Boids) will be uploaded in this [Github repository](#) .

1 Eulerian Systems: Computational Fluid Dynamics [50pt]

The first part of this assignment covers the subject of 2D Computational Fluid Dynamics from Practical 1. The task consists of designing, implementing, training and evaluating an autoregressive Transformer model on the 2D fluid vorticity dataset.

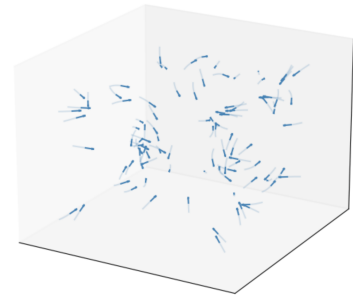


Task Description

- Train and evaluate the U-Net and the standard Transformer using their provided training and evaluation pipelines. You do not need to write any code.
- Implement the `PhysicsSliceAttention`, `PhysicsSliceBlock`, and `PhysicsSliceTransformer` classes according to the provided equations and tensor shapes.
- Train the physics-slice Transformer using the provided training pipeline and plot loss curves.
- Using the provided evaluation code, compare physics-slice Transformer's one-step MSE, relative L2 error, and autoregressive rollout errors with those of the standard Transformer and U-Net.
- Using the provided benchmarking code, compare the forward-pass speed of the physics-slice Transformer with the standard Transformer.
- Implement pushforward training for the physics-slice Transformer. Implement the complete pipeline, including dataset preparation, a scheduler for the number of pushforward steps, the training and validation procedures, and model evaluation. You may reuse or adapt code from previous sections where appropriate.
- Compare the pushforward-trained physics-slice Transformer with the same architecture trained without pushforward. Report and discuss their one-step MSE, one-step relative L_2 error, and autoregressive rollout errors.

2 Lagrangian Systems: Boids [50pt]

The second part of this assignment covers the subject of 3D Particle Dynamics from Practical 2. The task consists of designing, implementing, training and evaluating an autoregressive Graph Neural Network model on the Boids dataset.



Task Description

- Use the provided `BoidsDataset` class to load the data and implement the train/val/test splitting in the `DataModule`
- Think about what kinematic or interaction properties of the system might be interesting and plot them as exploratory data analysis
- Formulate and implement node and edge features that describe the state of the system in the `BoidsDataset` class.
- Design, implement and train a Graph Neural Network model with $E(3)$ -equivariant message passing in the `EGNN` and `BoidsModel` classes to predict the next state of the system based on only the previous state. Make sure that the code for autoregressive rollout during inference is correct in the `BoidsModel` class. Push-forward training is optional here and not required.
- Evaluate your model, plot loss curves, and show what it has and has not learned and hypothesize why. Investigate displacement errors, the kinematic and/or interactions properties or other performance measures and reflect on your node features, edge features and computational efficiency of the implemented architecture.

3 Deliverables

The assignment has 2 deliverables:

1. A `.zip` file, containing two Jupyter notebooks (part 1 and part 2) that both have run all the code cells and contain the required plots and results in the output cells. The `.zip` file should be uploaded in Ans. Explanations of the implementation or the results are not required within the notebook itself, they should be written in Ans. Code comments are not required, but appreciated.
2. In Ans, all assignment questions should be answered. These questions will be based on the task descriptions written above. We will ask you to upload the results, tables and/or plots that back up your explanations from the notebook, on Ans. The information put in Ans will be seen as ground truth, so make sure you fill in the latest and correct information, aligned with your notebook.